Reinforcing learning: Policy gradient method

YX Zhao
BJTU

November 6, 2025

Contents

1. Policy gradient method

2. Actor critic

Policy gradient method

Review DQN

deterministic policy in DQN

$a^* = \arg\max_a Q(s, a)$

In some case, probabilistic policy is optimal

game rock-paper-scissors

Solution

Policy gradient method

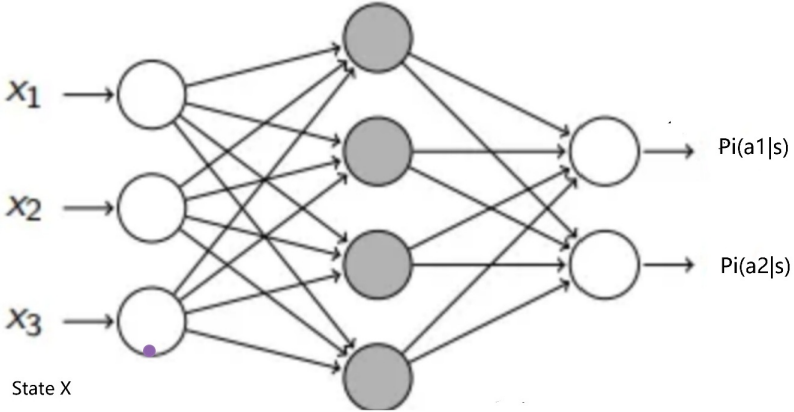
DQN vs Policy gradient

DQN estimate expectation return of $Q(s, a)$ while Policy gradient estimate optimal π directly which output action probability.

Represent policy

use neural network to represent policy $\pi(s)$

input is state, output is probability of each action



How about continuous action space

use case

auto-drive

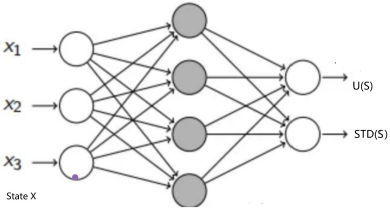
rebot

use continuous variable probability to represent action

use normal distribution to represent action probability

neural network output parameter of distribution, such as mean and std of normal distribution

use the distribution to generate action value



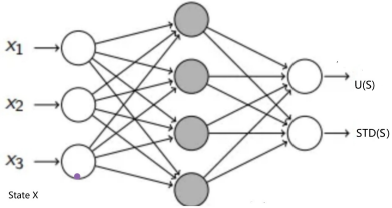
How to train policy gradient neural networks?

Review DQN

We can use bellman equation to generated target value. so we can train the network to close the target value

policy gradient neural networks

We don't know the optimal policy which is we want to find



advantage of action

Policy Gradient given policy $\pi_\theta(a|s)$

$$\bigtriangledown_\theta J(\theta) = E[A(s,a) \bigtriangledown_\theta \log \pi_\theta(s|a)] \tag{1}$$

advantage of action

where $A(s,a)$ is advantage of action a

$$E[A(s,a) = G(s,a) - V(s) \tag{2}$$
$$E[A(s,a) = Q(s,a) - V(s) \tag{3}$$
$$E[A(s,a) = r + \gamma V(s') - V(s) \tag{4}$$
$$\tag{5}$$

where $G(s,a) = r(t+1) + \gamma r(t+1) + \gamma^2 r(t+1) + \ldots$
$A(s,a)$ is how much better doing the action compared with average return $V(s))$

advantage of action

Policy Gradient given policy $\pi_\theta(a|s)$

$$\bigtriangledown_\theta J(\theta) = E[A(s,a) \bigtriangledown_\theta \log \pi_\theta(s|a)] \tag{6}$$

n step advantage of action

$$E[A(s,a) = r(t+1) + \gamma r(t+2) + \gamma^2 V(s''') - V(s) \tag{7}$$

$$\tag{8}$$

Math

trajectory Ω

$$\Omega = (s_1, a_1), (s_2, a_2), \ldots, (s_T, a_T) \tag{9}$$

$$G(\Omega) = \sum_{1}^{T} r(s_t, a_t) \tag{10}$$

object

$$J(\theta) = E(G(\Omega); \pi_\theta) \tag{11}$$

$$= \sum_{\Omega} p(\Omega; \theta) G(\Omega) \tag{12}$$

Math

object

$$J(\theta) = \sum_{\Omega} p(\Omega;\theta)G(\Omega) \tag{13}$$

$$\bigtriangledown_\theta J(\theta) = \bigtriangledown_\theta \sum_{\Omega} p(\Omega;\theta)G(\Omega) \tag{14}$$

$$= \sum_{\Omega} \bigtriangledown_\theta p(\Omega;\theta)G(\Omega) \tag{15}$$

$$= \sum_{\Omega} \frac{p(\Omega;\theta)}{p(\Omega;\theta)} \bigtriangledown_\theta \, p(\Omega;\theta)G(\Omega) \tag{16}$$

$$\tag{17}$$

Math

object

$$J(\theta) = \sum_{\Omega} \frac{p(\Omega;\theta)}{p(\Omega;\theta)} \bigtriangledown_\theta \, p(\Omega;\theta) G(\Omega) \tag{18}$$

$$= \sum_{\Omega} p(\Omega;\theta) \frac{\bigtriangledown_\theta p(\Omega;\theta)}{p(\Omega;\theta)} G(\Omega) \tag{19}$$

$$= \sum_{\Omega} p(\Omega;\theta) \bigtriangledown_\theta \log p(\Omega;\theta) G(\Omega) \tag{20}$$

$$= E(\bigtriangledown_\theta \log p(\Omega;\theta) G(\Omega)) \tag{21}$$

Math

gradient of trajectory return

gradient of trajectory return is calculated as following

$$\bigtriangledown_\theta J(\theta) = E(\bigtriangledown_\theta \log p(\Omega;\theta) G(\Omega)) \tag{22}$$

However, we can't use this expression to calculate $\bigtriangledown_\theta J(\theta)$ since there are infinite trajectories. We use samples to estimate it

$$\bigtriangledown_\theta J(\theta) = \frac{1}{M} \sum_{i=1}^{M} \bigtriangledown_\theta \log p(\Omega^i;\theta) G(\Omega^i)) \tag{23}$$

However, $p(\Omega^i;\theta)$ is still no good to calculate and we have to replace it

Math

probability of trajectory

according to markov chain rule, we have

$$p(\Omega;\theta) = \prod_{t=1}^{T} \pi_\theta(a_t|s_t)p(s_{t+1}|s_t,a_t) \tag{24}$$

$$\log p(\Omega;\theta) = \sum_{t=1}^{T} (\log \pi_\theta(a_t|s_t) + \log p(s_{t+1}|s_t,a_t)) \tag{25}$$

$\log p(s_{t+1}|s_t,a_t)$ is independent on θ, so

$$\bigtriangledown_\theta \log p(\Omega;\theta) = \sum_{t=1}^{T} (\bigtriangledown_\theta \log \pi_\theta(a_t|s_t)) \tag{26}$$

Finally, we can calculate this in pytorch

Math

probability of trajectory

Also, we have

$$G(\Omega) = \sum_{1}^{T} r(s_t, a_t) = \sum_{t=1}^{\tau} r(s_t, a_t) + \sum_{t=\tau+1}^{T} r(s_t, a_t) \tag{27}$$

where, τ is current time step, we only care second term of $G(\Omega)$ since what done is done. Thus we have

$$\bigtriangledown_\theta J(\theta) = E(\sum_{t=1}^{T}(\bigtriangledown_\theta \log \pi_\theta(a_t|s_t) \sum_{t=\tau+1}^{T} r(s_t, a_t)) \tag{28}$$

put math into code

objective

In summary, we have

$$\nabla_\theta J(\theta) = \frac{1}{M}\sum_{1=1}^{M}\frac{1}{T^i}\sum_{t=1}^{T^i}\nabla_\theta \log \pi_\theta(a_t^i|s_t^i)G^i(t) \tag{29}$$

Here M is number of trajectory and $T^i)$ is length i^{th} trajectory. We want to maximize $J(\theta)$, we use gradient ascend to update θ when we get a sample, which is

$$\theta = \theta + \eta \nabla_\theta \log \pi_\theta(a_t|s_t)G(t) \tag{30}$$

Basic idea behind this: if $G(t)$ is positive, we increase θ to increase the action's probability

put math into code

$\theta \leftarrow$ random
for *i=1:epoc-num* **do**
  $\Omega = ((s_1, a_1, r_1), (s_2, a_2, r_2), \ldots, (s_T, a_T, r_T))$=play a episode();
  for *t=1:T* **do**
    $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots$
  end
end

Algorithm 1: AC code

computing complex is T^2

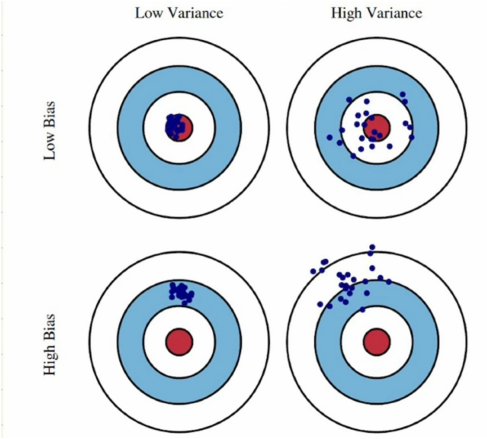
put math into code

G=0 **for** *t=T:-1:1* **do**
| $G = r_t + \gamma G$
end

Algorithm 2: calculate $G(t)$ efficiently

Actor critic

review bias vs variance



review gradient policy

ψ can be considered as a scale, gradient direction is determined by $\bigtriangledown_\theta \log \pi_\theta(a_t|s_t)$.
option of ψ:

$$\bigtriangledown_\theta J(\theta) = E\left[\sum_{t=1}^{T} \bigtriangledown_\theta \log \pi_\theta(a_t|s_t)\psi\right] \tag{31}$$

$$\psi = G(\Omega) = \sum_{t=1}^{T} \gamma^{t-1} r(s_t, a_t) \tag{32}$$

$$\psi = G(t) = \sum_{\tau=t+1}^{T} \gamma^{\tau-t-1} r(s_\tau, a_\tau) \tag{33}$$

$G(\Omega)$ compute once for a trajectory while $G(t)$ has to computer every step. $G(t)$ has lower variance since it does not use rewards in the past.
Both G have to wait trajectory finish which is a long time

Actor critic

Q value

$\psi = Q(s_t, a_t)$

we can update θ at each step, this method also decrease variance compared with former option

But we need a second neural networks to estimate $Q(s_t, a_t)$ and the second neural network is called as critic. The policy network is called as actor.

proof of replacing $G(t)$ with $Q(s_t, a_t)$ in the ψ

$$
\begin{aligned}
\bigtriangledown_\theta J(\theta) &= E\left[\bigtriangledown_\theta \log \pi_\theta(a_t|s_t) G(t)\right] && (34)\\
&= E\left[\bigtriangledown_\theta \log \pi_\theta(a_t|s_t) E\left[G(t)|s_t, a_t\right]\right] && (35)\\
&= E\left[\bigtriangledown_\theta \log \pi_\theta(a_t|s_t) Q(s_t, a_t)\right] && (36)\\
& && (37)
\end{aligned}
$$

advantage

Advantage can tell how much a action is better than average

$A(s_t, a_t) = G(t) - V(s)$
Reduce variance, low bias

replace G with Q

$A(s_t, a_t) = Q(s_t, a_t) - V(s)$
lower variance, need a addition neural network to estimate Q. $V(s)$ is easier to implement that $Q(s_t, a_t)$

TD,only $V(s)$

$A(s_t, a_t) = r + \gamma V(s') - V(s)$ TD(0)
$A(s_t, a_t) = r(t+1) + \gamma r(t+2) + \gamma^2 V(s(t+3)) - V(s)$ n-step balance variance and bias

AC implementation-1

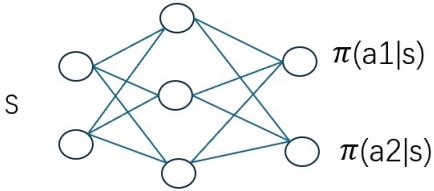


Figure: Policy networks

policy network's loss function

i represent i^{th} trajectory

$Loss_{political} = \frac{1}{M}\sum_{i=1}^{M} \log \pi_\theta(a^i|s^i)(r^{(i)} + \gamma(V(s'^{(i)}) - V(s^{(i)})))$

AC implementation-2

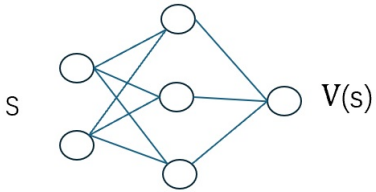


Figure: critic networks

critic's loss function

$Loss_{critical} = \frac{1}{M}\sum_{i=1}^{M} G^{(i)} - V_W(s^{(i)})$
where i represent i^{th} trajectory and W represent weight in critic networks
$G^(i) = r^{(i)} + V(s'^{(i)})$ we can also use n-step method

AC implementation-3

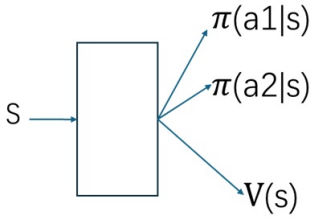


Figure: one networks

critic's loss function

$$Loss_{onenetwork} = Loss_{political} + \eta Loss_{critical}$$

entropy regularization

add entropy term in loss

$$Loss_{onenetwork} = Loss_{political} + \eta Loss_{critical} + \beta(-entropy(\pi))$$

experiment

E:\lecture_code\AI\rl\A2C_Cartpole.ipynb

advance experiment

E:\lecture_code\AI\rl\A2C_Atari.ipynb

advance experiment

use DDQN to realize a atari video game